

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра математического обеспечения и применения ЭВМ

ОТЧЕТ
по лабораторной работе №3
по дисциплине «Построение и анализ алгоритмов»
Тема: Динамическое программирование

Студент гр. 4345

Гаврилов Е.В.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

Цель работы

Изучить и реализовать алгоритм поиска расстояния Левенштейна и алгоритм нахождения длины наибольшей общей подпоследовательности двух строк с помощью динамического программирования.

Задание

Расстоянием Левенштейна назовём минимальное количество операций вставки одного символа, удаления одного символа и замены одного символа на другой, необходимых для превращения одной строки в другую.

Разработайте программу, осуществляющую поиск расстояния Левенштейна между двумя строками.

Пример

Для строк `pedestal` и `stien` расстояние Левенштейна равно 7:

- Сначала нужно совершить четыре операции удаления символа: `pedestal` -> `stal`.
- Затем необходимо заменить два последних символа: `stal` -> `stie`.
- Потом нужно добавить символ в конец строки: `stie` -> `stien`.

Параметры входных данных

Первая и вторая строка входных данных содержит строки S , T из строчных латинских букв (S , $1 \leq |S| \leq 2550$), (T , $1 \leq |T| \leq 2550$).

Параметры выходных данных

Одно число L , равное расстоянию Левенштейна между строками S и T .

Sample Input

```
pedestal
stien
```

Sample Output

```
7
```

Вариант 14a

Методом динамического программирования вычислить длину наибольшей общей подпоследовательности двух строк.

Выполнение работы

В ходе работы с помощью динамического программирования были реализованы алгоритм вычисления расстояния Левенштейна и алгоритм нахождения длины наибольшей общей подпоследовательности (LCS).

Описание функции `levenshteinDistance(s, t string) int`

На вход подаются две строки s и t . Функция возвращает целое число: минимальное количество операций вставки, удаления и замены символа, необходимых для превращения строки s в строку t .

Для начала вычисляются длины строк $m = \text{len}(s)$ и $n = \text{len}(t)$. Создаётся матрица dp размером $(m+1)*(n+1)$, где $dp[i][j]$ это минимальное расстояние Левенштейна между префиксом s длины i и префиксом t длины j .

Происходит инициализация базовых случаев:

- $dp[i][0] = i$ для всех i от 0 до m
- $dp[0][j] = j$ для всех j от 0 до n

Далее начинается процесс заполнения матрицы. Для каждого i от 1 до m и для каждого j от 1 до n :

- Если символы $s[i-1]$ и $t[j-1]$ совпадают, то $dp[i][j] = dp[i-1][j-1]$, так как дополнительная операция не требуется.
- Если символы не совпадают, то $dp[i][j] = \min(dp[i-1][j] + 1, dp[i][j-1] + 1, dp[i-1][j-1] + 1)$, где:
 - $dp[i-1][j] + 1$ соответствует операции удаления символа из s
 - $dp[i][j-1] + 1$ соответствует операции вставки символа в s
 - $dp[i-1][j-1] + 1$ соответствует операции замены символа

По завершении всех итераций функция возвращает значение $dp[m][n]$, которое и будет расстояние Левенштейна.

Описание функции `longestCommonSubsequence(s, t string) int`

На вход подаются строки s и t . Функция возвращает длину наибольшей общей подпоследовательности (LCS) строк s и t .

В процессе инициализации создается и инициализируется нулями матрица dp размером $(m+1)*(n+1)$, где $m = \text{len}(s)$, $n = \text{len}(t)$.

Затем происходит заполнение матрицы. Для каждого i от 1 до m и для каждого j от 1 до n :

- Если символы $s[i-1]$ и $t[j-1]$ совпадают, то $dp[i][j] = dp[i-1][j-1] + 1$ (добавление текущего совпадающего символа к LCS префиксов).
- Если символы не совпадают, то $dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$, то есть выбирается максимальная длина LCS: без учёта текущего символа s или без учёта текущего символа t .

По завершении всех итераций функция возвращает $dp[m][n]$ — длину наибольшей общей подпоследовательности исходных строк.

Сложность алгоритма

Сложность алгоритма вычисления расстояния Левенштейна, где входные данные это m — длина первой строки, n — длина второй строки:

по времени: $O(m*n)$, поскольку для заполнения матрицы dp размером $(m+1)*(n+1)$ требуется выполнить $m*n$ итераций, на каждой из которых выполняется константное количество операций: сравнение символов, вычисление минимума из трёх значений и присваивание.

по памяти: $O(m*n)$, так как создаётся и хранится матрица dp размером $(m+1)*(n+1)$. Каждый элемент матрицы хранит текущее расстояние Левенштейна для соответствующего префикса. Алгоритм может быть оптимизирован до $O(\min(m, n))$ путём использования только двух строк матрицы (текущей и предыдущей), поскольку для вычисления очередного значения требуются только значения из текущей и предыдущей строки.

Исходный код программы см. в приложении А.

Результаты тестирования функции нахождения расстояния Левенштейна см. в Таблице 1.

Результаты тестирования функции нахождения длины наибольшей общей подпоследовательности см. в Таблице 2.

Таблица 1 – Тестирование функции расстояния Левенштейна

№	Ввод	Вывод	Комментарий
1	lololol lololol	0	Одинаковые строки
2	abc ab	1	Удаление одного символа
3	abc abd	1	Замена одного символа
4	abc abcd	1	Добавление одного символа
5	abc lol	3	Все символы разные
6	pedestal stien	7	Тест из примера

Таблица 2 – Тестирование функции нахождения длины LCS

№	Ввод	Вывод	Комментарий
1	abc abc	3	Строки совпадают
2	abc lol	0	Все символы разные
3	abcd acbd	3	Два возможных варианта LCS длины 3 (acd, abd)
4	abcde edcba	1	Строки перевернуты (LCS это один любой символ)

Вывод

Реализованы динамические алгоритмы нахождения расстояния Левенштейна и нахождения LCS. Дана асимптотическая оценка сложности их реализации по времени: $O(m*n)$, памяти: $O(m*n)$, где m — длина первой строки, n — длина второй строки. Алгоритмы протестированы и отлажены.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Файл main.go

```
package main

import (
    "bufio"
    "fmt"
    "os"
)

func main() {
    var s, t string
    reader := bufio.NewReader(os.Stdin)
    fmt.Fscan(reader, &s)
    fmt.Fscan(reader, &t)

    distance := levenshteinDistance(s, t)
    fmt.Println("Resulting Levenshtein distance is:", distance)
    lengthLCS := longestCommonSubsequence(s, t)
    fmt.Println("Resulting length of longest common subsequence is:",
lengthLCS)
}

func printMatrixWithStrings(matrix [][]int, s, t string) {
    fmt.Printf("  %s\n", t)
    for i := range matrix {
        if i >= 1 {
            fmt.Printf("%c", s[i-1])
        } else {
            fmt.Print(" ")
        }
        for j := range matrix[i] {
            fmt.Print(matrix[i][j])
        }
        fmt.Println()
    }
    fmt.Println()
}

func printMatrix(matrix [][]int) {
    for i := range matrix {
        for j := range matrix[i] {
            fmt.Print(matrix[i][j])
        }
        fmt.Println()
    }
    fmt.Println()
}

func levenshteinDistance(s, t string) int {
    m, n := len(s), len(t)
    dp := make([][]int, m+1)
```

```

    for i := range dp {
        dp[i] = make([]int, n+1)
    }

    fmt.Printf("Calculating Levenshtein distance between the strings
%s and %s\n", s, t)
    for i := 0; i <= m; i++ {
        dp[i][0] = i
    }
    for j := 0; j <= n; j++ {
        dp[0][j] = j
    }
    fmt.Println("Initial matrix of operations:")
    printMatrixWithStrings(dp, s, t)

    fmt.Printf("Starting calculations\n\n")
    for i := 1; i <= m; i++ {
        for j := 1; j <= n; j++ {
            fmt.Printf("Now on position (%d, %d)\n", i, j)
            fmt.Printf("In the strings, position maps to symbols:
%c, %c\n", s[i-1], t[j-1])
            if s[i-1] == t[j-1] {
                dp[i][j] = dp[i-1][j-1]
                fmt.Println("Symbols match! The resulting value
for the position is ", dp[i][j])
                fmt.Println("\tTook action MATCH")
                fmt.Println("Now matrix looks like:")
                printMatrix(dp)
            } else {
                dp[i][j] = min(
                    dp[i-1][j]+1,
                    dp[i][j-1]+1,
                    dp[i-1][j-1]+1,
                )
                fmt.Println("Symbols don't match! The resulting
value for the position is ", dp[i][j])
                if dp[i][j] == dp[i-1][j]+1 {
                    fmt.Println("\tTook action DELETE (or other
action, if listed)")
                }
                if dp[i][j] == dp[i-1][j-1]+1 {
                    fmt.Println("\tTook action REPLACE (or
other action, if listed)")
                }
                if dp[i][j] == dp[i][j-1]+1 {
                    fmt.Println("\tTook action INSERT (or other
action, if listed)")
                }
                fmt.Println("Now matrix looks like:")
                printMatrix(dp)
            }
        }
    }

    return dp[m][n]
}

```



```

func longestCommonSubsequence(s, t string) int {
    m, n := len(s), len(t)

    dp := make([][]int, m+1)
    for i := range dp {
        dp[i] = make([]int, n+1)
    }

    fmt.Printf("Calculating LCS of the strings %s and %s\n", s, t)

    for i := 1; i <= m; i++ {
        for j := 1; j <= n; j++ {
            fmt.Printf("Now on position (%d, %d)\n", i, j)
            fmt.Printf("In the strings, position maps to symbols:
%c, %c\n", s[i-1], t[j-1])
            if s[i-1] == t[j-1] {
                dp[i][j] = dp[i-1][j-1] + 1
                fmt.Println("Symbols match! The resulting length
of LCS for the prefix is ", dp[i][j])
            } else {
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])
                fmt.Println("Symbols don't! The resulting length
of LCS for the prefix is ", dp[i][j])
            }
            fmt.Println("Now matrix looks like:")
            printMatrix(dp)
        }
    }

    return dp[m][n]
}

```